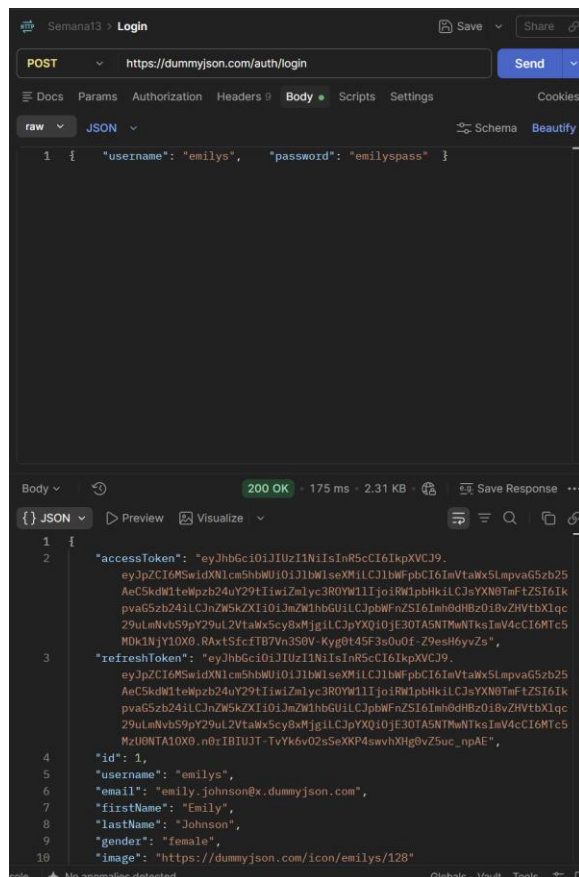






2. Utilizar token

- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: <https://dummyjson.com/user/me>
- En Authorization seleccione Bearer Token.
- Copie el accessToken obtenido en la solicitud Login.
- Ejecute la solicitud.
- Verifique que la API devuelve la información del usuario autenticado.

The image shows two side-by-side screenshots of a REST client interface. Both screenshots show a GET request to <https://dummyjson.com/user/me> with the Authorization header set to Bearer Token. The response status is 200 OK. The response body is a JSON object containing user details.

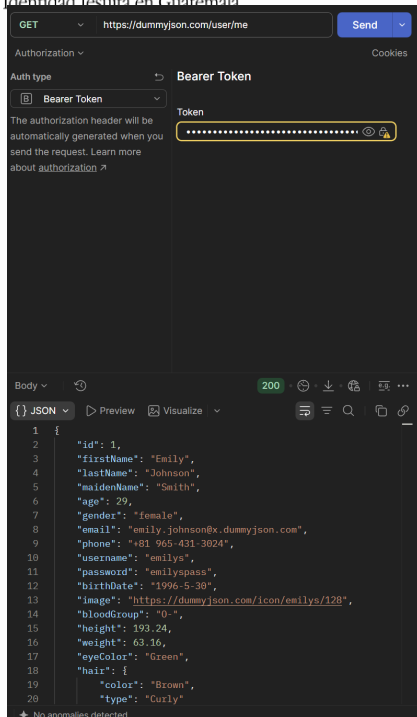
```
{  "id": 1,  "firstName": "Emily",  "lastName": "Johnson",  "maidenName": "Smith",  "age": 29,  "gender": "female",  "email": "emily.johnson@x.dummyjson.com",  "phone": "+81 965-431-3824",  "username": "emilys",  "password": "emilyspass",  "birthDate": "1996-5-30",  "image": "https://dummyjson.com/icon/emilys/128",  "bloodGroup": "O-",  "height": 193.24,  "weight": 63.16,  "eyeColor": "Green",  "hair": {    "color": "Brown",    "type": "Curly"  }}
```

3. Comprobar qué ocurre sin autenticación

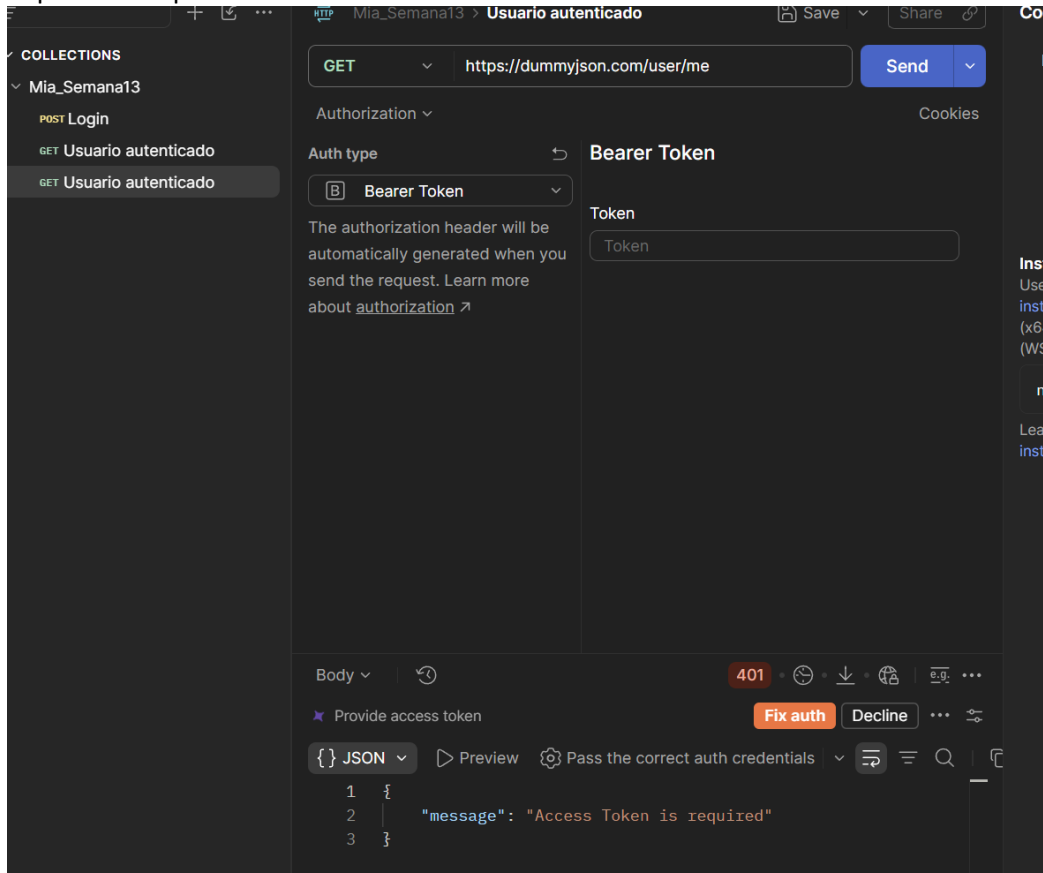
- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.
- Compare la respuesta con la solicitud autenticada.
- Registre el código HTTP y explique la diferencia.

4. Evidencia

- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.



- Captura de la prueba sin token.



- Indique qué función cumple el Bearer Token.
Sirve para autenticar las solicitudes HTTP y otorgar acceso a los recursos protegidos de una API sin necesidad de enviar credenciales

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.



Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- Abra el proyecto C# de Azure Storage.
- En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- Busque la variable que contiene la Connection String.
- Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada:

AZURE_STORAGE_CONNECTION_STRING

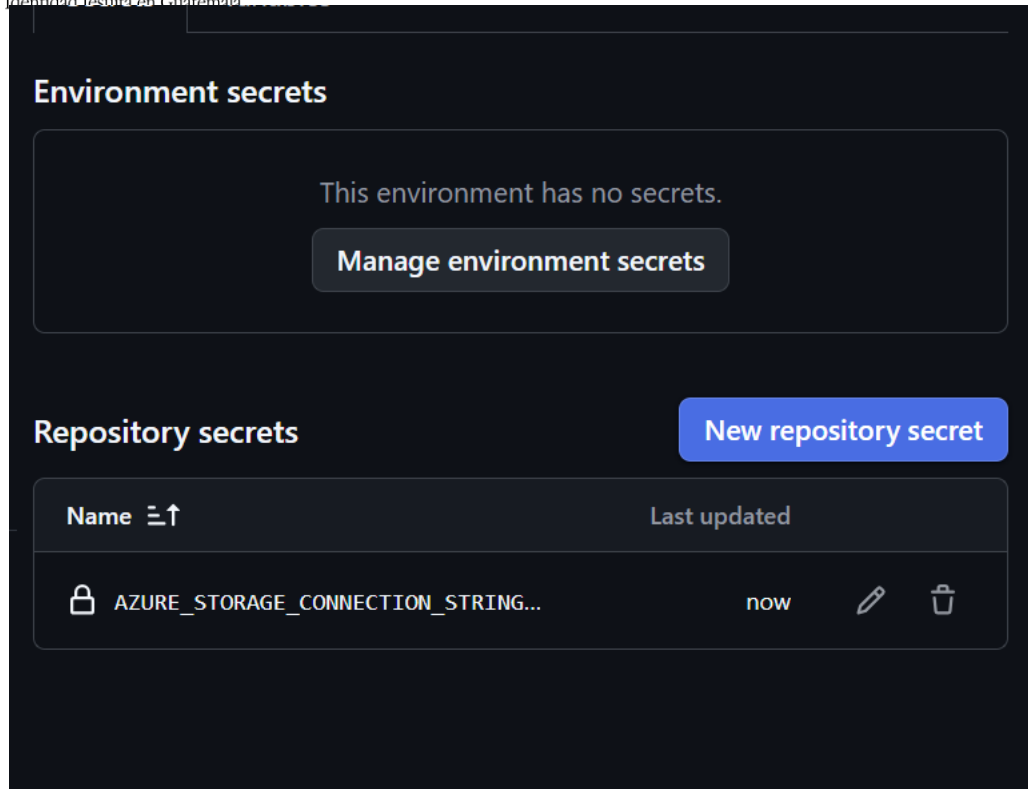
La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

7. Crear la variable para ejecutar localmente

- En la carpeta principal del proyecto, cree un archivo llamado .env.
- Dentro del archivo agregue la variable:
`AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING`
- Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- Compruebe que el programa continúa funcionando correctamente.
- No publique el archivo .env en GitHub**, ya que contiene la credencial real.

8. Crear el Secret en Github

- Ingresa al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: `AZURE_STORAGE_CONNECTION_STRING`
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.